ENGS 101 Final Project

Red–Blue Cybersecurity RL Challenge

Instructor: Prof. Peter Chin
Intermediate: November 1, 2026, 23:59 ET
Final: November 17, 2026, 23:59 ET

- ◇ **This is a group project.** Each group should designate one representative to submit on behalf of the group. Other group members do not need to submit separately.
- ◇ The representative uploads a ZIP at `arena.codefly.run`. Name the archive with the representative's seven-character Dartmouth student ID. Use the same representative ID for all versions and both stages.
- ◇ Submit one agent implementation. The same code must support both the Red and Blue roles.
- ◇ You may use the Python standard library, PyTorch, and NumPy. You may not use an existing reinforcement-learning implementation or framework.
- ◇ The project has two separately recorded stages: an Intermediate submission and a Final submission. Both stages use the same upload format and interface.
- ◇ The course server performs the official training and evaluation. Local results are for development only.
- ◇ **Unlimited resubmissions are allowed before each stage's deadline.** Your representative may upload a version, view its automatic evaluation results and points, improve the group's agent, and submit another version before the deadline.

1. Assessment Schedule and Grading

Dartmouth's Fall 2026 classes begin on September 14 and end on November 17. The Intermediate deadline is the end of Week 7, November 1, 2026 at 23:59 Eastern Time. The Final deadline is the last day of classes, November 17, 2026 at 23:59 Eastern Time. These dates are based on the Registrar's Fall Term 2026 calendar: Dartmouth Fall Term 2026 calendar.

The server preserves each upload as a separate timestamped source snapshot. Please select the correct stage on the upload page, use your group's designated representative's student ID, and do not share the submission website outside the class. There is no limit on the number of submissions before the relevant deadline. Each upload contains one current version of the group's source code; the server trains fresh Red and Blue models for that version. Check the displayed results, make improvements and resubmit as needed. Automatic performance points are feedback; staff confirm course grades after reviewing the required RL implementation. Pending review is not a deduction, and no additional report is required for that review.

Each stage is scored out of 100 points. An upload receipt confirms only the archive format. Both role-training runs, model reload and evaluation must succeed before stage credit is recorded. Fix a failed run and resubmit. The role score is compared with the fixed course teacher baseline; a role "beats" a baseline when its fixed-evaluation point estimate is strictly above that baseline.

Stage result	**Stage score**
Intermediate: both Red and Blue beat the teacher baseline	100
Intermediate: exactly one role beats the teacher baseline	80
Intermediate: submission passes validation but neither role beats the baseline	60
Final: both roles beat the Final teacher challenge	100
Final: exactly one role beats the Final teacher challenge	80
Final: neither role beats the Final challenge, but both roles beat the Intermediate teacher baseline	60
Final: neither role beats the Final challenge, and exactly one role beats the Intermediate baseline	40
Final: neither role beats either teacher baseline	20

After the Intermediate deadline, the course team freezes the valid Intermediate policies and trains a stronger Final teacher challenge from that cohort. The Final teacher challenge, hidden scenarios, random seeds, training budget and evaluation budget are fixed before Final submissions are evaluated. The student-vs-student matrix is a diagnostic leaderboard and does not determine the stage score.

Final uploads remain closed until the Final teacher challenge is frozen and released by the course team. Intermediate and Final points are retained separately even when a newer upload replaces the active leaderboard entry.

2. Objective

Design and implement a reinforcement-learning agent for a simulated cybersecurity environment. Red attempts to discover, compromise, and impact network assets. Blue attempts to detect activity, deploy decoys, remove access, and restore hosts.

Your code will be trained separately as Red and Blue. The resulting frozen policies will be scored relative to the frozen course baseline. They will also play against the policies submitted by the rest of the class in a separate cross-play matrix.

3. Starter repository

Clone the public repository:

```
git clone https://github.com/cccat6/engs101-cyber-rl
cd engs101-cyber-rl
python -m venv .venv
```

Activate `.venv` using the command for your operating system, then run:

```
python -m pip install -r requirements.txt
```

The repository contains a small public environment and a complete runnable example. The student-facing files are:

File	Purpose
`student_agent.py`	Implement your agent here. This is the main file you will edit.
`run.py`	Train and test the same agent locally as both Red and Blue. Keep this file in the submission.
`scenarios/base.yaml`	The public local environment used by `run.py`.

Python 3.11 or later is required. Install the dependencies from `requirements.txt`. The file contains PyTorch, NumPy, PyYAML, and the local environment package.

The public environment is smaller than the hidden server environment. Do not hard-code observation dimensions, action counts, host names, or a particular topology.

4. Required interface

Keep the following function:

```
def create_agent(config):
    return YourAgent(config)
```

The returned object must expose:

```
agent.model
agent.reset(seed)
agent.act(observation, training)
agent.learn(transition)
```

`agent.model` must be a `torch.nn.Module`. The model may contain at most 1,000,000 registered parameters per role. The local runner and the course server perform the same check immediately after `create_agent(config)` returns. A submission that exceeds the limit is rejected before environment training begins.

The other three members have the following meanings:

Member	Purpose
`reset(seed)`	Reset episode-local state.
`act(observation, training)`	Return one valid integer action.
`learn(transition)`	Store the transition and perform any training update.

The runner supplies this configuration:

```
config.role
config.observation_dim
config.num_actions
config.total_steps
config.seed
config.device
```

Always construct the model from the supplied dimensions and move it to `config.device`. The server normally supplies a CUDA device; local development may use CPU, CUDA, Intel XPU, or Apple MPS.

5. Observations and transitions

The policy observation has two fields:

```
observation.features
observation.action_mask
```

`features` is a one-dimensional `float32` NumPy array. `action_mask` is a Boolean array with one entry per action. `act` must return an action whose mask entry is true.

During training, `learn` receives:

```
transition.observation
transition.action
transition.reward
transition.next_observation
transition.done
```

The runner calls `learn` exactly once per environment step. Evaluation does not call `learn`.

6. Implementation rule

You must implement the RL algorithm yourself using PyTorch tensor operations, neural-network modules, and optimizers. Allowed components include `torch.nn.Linear`, activation functions, automatic differentiation, and `torch.optim`.

Do not import an implementation of DQN, PPO, actor–critic, replay buffers, training loops, or policies from an RL package. In particular, do not use Stable-Baselines3, RLlib, CleanRL, Tianshou, TorchRL algorithms, or equivalent ready-made implementations.

7. Local run

From the repository root, run:

```
python run.py --steps 1000 --device auto
```

This single command:

1. trains the submitted code as Red;
2. saves and reloads the Red model;
3. tests the frozen Red model;
4. repeats the same process as Blue.

After testing your changes, package only the required source files:

```
python package_submission.py YOUR_ID
# If your agent imports a helper module:
python package_submission.py YOUR_ID --include helper.py
```

Replace `YOUR_ID` with your group's designated representative's seven-character Dartmouth ID.

The provided `SimpleAgent` is a minimal one-layer policy-gradient example. It runs end to end and updates its PyTorch model, but it is deliberately small and is not intended to be competitive. You may replace its implementation while preserving the required interface.

8. Server training and evaluation

The course server supplies the official environment-step budget, random seeds, training scenarios, device, and evaluation episodes. These values are the same for all submissions.

For each role, the runner uses one frozen, course-owned teacher policy as the opponent. The public repository contains teacher checkpoints for local work; the server uses separately trained private checkpoints with the same interface. The teacher opponent is reset between episodes but is never updated during student training. Thus every submission receives the same number of environment steps and the same fixed opponent.

After training, the server saves the weights in `agent.model`. It then creates a fresh agent, reloads those weights, and performs frozen inference.

The public and hidden environments use the same interface but may have different dimensions and network topologies.

9. Reward and score

The environment supplies a per-step reward for learning. The leaderboard score is computed separately from the mission loss over the whole episode.

For an episode of length T, let

$$D = \frac{1}{T} \sum_{t=1}^{T} \text{mission_loss}_t.$$

The corresponding role scores are

$$\text{RedScore} = 100D, \qquad \text{BlueScore} = 100(1 - D).$$

For the stable score, each frozen student role is evaluated against the opposite frozen teacher role. Let R_s be the raw score of a student Red policy against teacher Blue, and let R_b be teacher Red's raw score against teacher Blue. Define the relative Red score as

$$\text{RelativeRed} = \text{clip}(50 + R_s - R_b, 0, 100).$$

The relative Blue score is defined analogously from student Blue against teacher Red. The teacher baseline therefore receives 50 for each role. The stable score is the average of the relative Red and Blue scores. The server also reports a 95% confidence interval. This centering changes only the aggregate, not the environment reward or any episode score. Training rewards are not leaderboard points.

10. Leaderboard

Each accepted submission produces one frozen Red policy and one frozen Blue policy. The main standing ranks the stable score relative to the teacher baseline. The server separately

evaluates every submitted Red policy against every submitted Blue policy and the course baseline.

When a new submission is accepted, the server recomputes baseline-relative scores, rebuilds the comparison matrix using the saved frozen policies, and re-ranks all participants. Old policies are re-evaluated, not retrained. The upload form, submission status, stable standings, confidence intervals, and Red–Blue comparison matrix appear on the same course webpage. Matrix scores are relative to the current class population and do not determine the stable score. Hidden scenarios and trajectories are not released.

11. Submission

Only one designated representative needs to upload on behalf of each group. Upload one source-code ZIP per version through the course website. Its filename must be the representative's seven-character student ID followed by `.zip`. Use that same ID for all resubmissions and both stages. The ID may contain ASCII letters and digits, in either case; the server converts it to lowercase. For example:

```
F001ABC.zip
```

Submit only the runnable entry points and any Python helper modules used by your agent. The archive may place these source files at its root:

```
F001ABC.zip
|-- run.py
|-- student_agent.py
`-- helper.py                 (if used)
```

Alternatively, the same contents may be inside one folder:

```
F001ABC.zip
`-- F001ABC/
    |-- run.py
    |-- student_agent.py
    `-- helper.py             (if used)
```

If the archive contains a single outer folder, that folder must have the same student ID as the ZIP filename, ignoring case. After normalization, `run.py` and `student_agent.py` must be at the submission root. Every submitted file must be Python source with a `.py` extension. The ZIP file and its extracted Python source must each be no larger than 100 KiB. An archive that does not follow these rules may be rejected or removed from the submission system. Submit a corrected archive using your designated representative's seven-character

student ID in the filename. Do not include the starter repository, scenarios, dependency files, `.venv`, local checkpoints, pretrained weights, or other generated files. A malformed or oversized archive is rejected by the website with an error message.

Do not submit pretrained weights. The server trains a fresh Red model and a fresh Blue model from your code. `run.py` remains in the archive so the submission is directly runnable locally. Official training calls `create_agent` in `student_agent.py`.

Submissions must:

1. import successfully in the fixed course environment;
2. follow the required interface;
3. return only valid actions;
4. use only the permitted libraries;
5. keep each role's model at or below 1,000,000 registered parameters;
6. complete within the server resource limits.

The official server training budget and evaluation episode count are controlled by the course evaluator and are shown on the upload page. The current budget is 10,000 environment steps per role and five evaluation episodes per hidden scenario. Deadline checks use the receipt time, not when queued grading finishes. Both deadlines are in `America/New_York`; the UTC offset is −05:00 on November 1 and November 17. The two stage rules and deadlines above are the published project grading policy.